LAPORAN PRAKTIKUM
STRUKTUR DATA
QUEUE

Disusun oleh:

Ghinada Fathanawafa Algma
2511533008

Dosen Pengampu:
Dr. Wahyudi S.T. M.T

Asisten Praktikum:
Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur atas kehadirat Tuhan Yang Maha Esa, berkat Rahmat dan karunia-Nya laporan praktikum Struktur Data tentang Queue menggunakan bahasa Java ini dapat saya selesaikan dengan baik. Laporan ini saya susun sebagai bentuk pertanggungjawaban atas praktikum yang telah saya laksanakan.

Dalam penyusunan laporan ini saya banyak mendapat bimbingan serta dukungan dari beberapa pihak, oleh karena itu saya mengucapkan terima kasih kepada dosen pengampu, asisten praktikum dan rekan rekan praktikan yang turut membantu dalam pelaksanaan kegiatan praktikum.

Saya menyadari bahwa lapporan praktikum ini jauh dari sempurna. Oleh karena itu kritik dan saran yang membangun sangat berguna untuk perkembangan lebih baik kedepannya. Semoga laporan praktikum ini dapat bermanfaat bagi saya serta pembaca, khususnya dalam memahami konsep Queue.

Padang, April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR i

DAFTAR ISI ii

BAB I PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Tujuan 1

1.3 Manfaat 1

BAB II PEMBAHASAN 2

2.1 Dasar Teori 2

2.1.1 Queue 2

2.1.2 Operasi dasar pada Queue 2

2.1.3 Implementasi Queue 2

2.1.4 Pemrograman dengan Queue 2

2.2 Kode Program dan Penjelasan 3

2.2.1 Class QueueArray_2511533008 3

2.2.2 Class QueueArrayDriver_2511533008 7

2.2.3 Class QueueLinkedList_2511533008 8

2.2.4 ReverseData_2511533008 9

2.2.5 Class IterasiQueue_2511533008 10

2.3 Output Program 12

2.3.1 Output QueueArrayDriver_2511533008 12

2.3.2 Output QueueLinkedList_2511533008 12

2.3.3 Output ReverseData_2511533008 12

2.3.4 Output IterasiQueue_2511533008 13
BAB III PENUTUP 14
3.1 Kesimpulan 14
DAFTAR PUSTAKA 15
TUGAS 16
1. Kode Program 16
1.1 Class AntrianLoket_2511533008 16
1.2 Class Queue_2511533008 17
2. Output Program 19

BAB I
PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu aspek fundamental dalam ilmu komputer yang berfungsi untuk mengatur dan mengelola data agar dapat diproses secara efisien. Salah satu bentuk struktur data linear yang penting adalah Queue, yang bekerja dengan prinsip *First In First Out (FIFO)*, yaitu elemen pertama yang dimasukkan akan menjadi elemen pertama yang dikeluarkan. Queue bersifat antrian.

Praktikum ini penting karena Queue banyak digunakan dalam berbagai sistem komputer maupun kehidupan sehari-hari, seperti pada antrean di bank, sistem penjadwalan CPU, antrean printer, hingga algoritma pencarian graf seperti *Breadth First Search*. Melalui praktikum ini, kita dilatih untuk mengimplementasikan operasi dasar Queue seperti *enqueue* (menambahkan data), *dequeue* (menghapus data), dan *peek* (melihat elemen terdepan), serta memahami kondisi *isEmpty* dan *isFull*. Selain itu, praktikum ini juga memperkenalkan berbagai cara implementasi, baik menggunakan array, circular queue, maupun linked list, sehingga kita dapat membandingkan kelebihan dan kekurangan masing-masing metode.

1.2 Tujuan

1. Memahami prinsip FIFO dalam pengolahan data.
2. Melatih implementasi operasi dasar pada Queue : *enqueue, dequeue, peek.*
3. Mempelajari validasi kondisi : *isFull, isEmpty.*
4. Bisa mengimplementasikan Queue dengan Array ataupun LinkedList.

1.3 Manfaat

1. Memperkuat pemahaman terkait materi Queue karena diimplementasikan langsung.
2. Melatih pemahaman tentang kode program.

BAB II
PEMBAHASAN

2.1 Dasar Teori

2.1.1 Queue

Queue merupakan konsep dasar ilmu komputer yang digunakan untuk menyimpan dam mengelola data dalam urutan tertentu. Queue menggunakan prinsip *FIFO (First In First Out)*. Dimana data yang pertama kali masuk merupakan data yang pertama kali keluar. Queue biasanya digunakan dalam berbagai algoritma dan aplikasi karena sederhana dan efisien dalam mengolah data. Queue beroperasi seperti garis di mana elemen ditambahkan pada satu ujung (rear) dan dihapus dari ujung lainnya (front).

2.1.2 Operasi dasar pada Queue

1. *Enqueue* (sisipan) : Menambahkan elemen ke bagian belakang queue.
2. *Dequeue* (hapus) : menghapus dan mengembalikan elemen dari bagian depan queue.
3. *Peek* : return elemen dibagian depan queue tanpa menghapusnya.
4. *Empty* : memeriksa apakah queue kosong.
5. *Full* : memeriksa apakah queue sudah penuh.

2.1.3 Implementasi Queue

Queue (antrian) diimplementasikan dengan menggunakan 2 teknik :

1. Implementasi dengan menggunakan Array
2. Implementasi dengan menggunakan LinkedList

2.1.4 Pemrograman dengan Queue

1. add(value)
2. remove()
3. peek()
4. size()
5. isEmpty()

2.2 Kode Program dan Penjelasan

2.2.1 Class QueueArray_2511533008

Kode Program :

```
package pekan4_2511533008;

public class QueueArray_2511533008 {
	int front_3008, rear_3008, size_3008;
	int capacity_3008;
	int array_3008[];

	public QueueArray_2511533008(int capacity_3008) {
		this.capacity_3008 = capacity_3008;
		front_3008 = this.size_3008 = 0;
		rear_3008 = capacity_3008 - 1;
		array_3008 = new int[this.capacity_3008];
	}

	boolean isFull_3008(QueueArray_2511533008 queue_3008) {
		return (queue_3008.size_3008 ==
queue_3008.capacity_3008);
	}

	boolean isEmpty_3008(QueueArray_2511533008 queue_3008) {
		return (queue_3008.size_3008 == 0);
	}

	void enqueue_3008(int item_3008) {
		if (isFull_3008(this))
			return;
		this.rear_3008 = (this.rear_3008 + 1) %
this.capacity_3008;
		this.array_3008[this.rear_3008] = item_3008;
		this.size_3008 = this.size_3008 + 1;
		System.out.println(item_3008 + " enqueue to queue");
	}

	int dequeue_3008() {
		if (isEmpty_3008(this))
			return Integer.MIN_VALUE;
		int item_3008 = this.array_3008[this.front_3008];
		this.front_3008 = (this.front_3008 + 1) %
this.capacity_3008;
		this.size_3008 = this.size_3008 - 1;
		return item_3008;
	}
	int front_3008() {
		if (isEmpty_3008(this))
			return Integer.MIN_VALUE;
```

```
            return this.array_3008[this.front_3008];
        }
        int rear_3008() {
            if (isEmpty_3008(this))
                return Integer.MIN_VALUE;

            return this.array_3008[this.rear_3008];
        }

        //mencetak elemen antrian
        void display_3008() {
            int i_3008;
            if (front_3008 == rear_3008) {
                System.out.printf("\nAntrian Kosong");
                return;
            }
            //kunjungi dari belakang dan cetak
            for (i_3008 = front_3008; i_3008 < rear_3008; i_3008++)
{
                System.out.printf("  %d <--",
array_3008[i_3008]);
            }
            return;
        }

}
```

Penjelasan :

Pada class QueueArray_2511533008 ini merupakan program adt nya yang akan dipakai untuk menjalankan class QueueArrayDriver_2511533008. Pada program ini kita mengggunakan array untuk mengimplementasikan queue.

1. Struktur pada class

```
public class QueueArray_2511533008 {
    int front_3008, rear_3008, size_3008;
    int capacity_3008;
    int array_3008[];
```

Gambar 2.2.1.1 struktur class

front_3008 : posisi elemen paling depan

rear_3008 : posisi elemen paling belakang

size_3008 : jumlah elemen saat dalam queue

capacity_3008 : kapasitas maksimum queue

array_3008[] : array untuk menyimpan data queue

2. Konstruktor

```
public QueueArray_2511533008(int capacity_3008) {
    this.capacity_3008 = capacity_3008;
    front_3008 = this.size_3008 = 0;
    rear_3008 = capacity_3008 - 1;
    array_3008 = new int[this.capacity_3008];
}
```

Gambar 2.2.1.2 konstruktor

Konstruktor ini berfungsi untuk menginisialisasi queue dengan kapasitas tertentu.

front_3008 dan size_3008 dimulai dari 0

rear_3008 di set menjadi capacity_3008 – 1 agar saat pertama kali enqueue.

Array_3008 dibuat sesuai dengan kapasitas.

3. Pengecekan kondisi

```
boolean isFull_3008(QueueArray_2511533008 queue_3008) {
    return (queue_3008.size_3008 == queue_3008.capacity_3008);
}

boolean isEmpty_3008(QueueArray_2511533008 queue_3008) {
    return (queue_3008.size_3008 == 0);
}
```

Gambar 2.2.1.3 pengecekan kondisi

isFull_3008 bernilai true jika jumlah elemen sama dengan kapasitas.

isEmpty_3008 bernilai true jika jumlah elemen = 0.

4. Enqueue (menambah elemen)

```
void enqueue_3008(int item_3008) {
    if (isFull_3008(this))
        return;
    this.rear_3008 = (this.rear_3008 + 1) % this.capacity_3008;
    this.array_3008[this.rear_3008] = item_3008;
    this.size_3008 = this.size_3008 + 1;
    System.out.println(item_3008 + " enqueue to queue");
}
```

Gambar 2.2.1.4 enqueue

Saat menambahkan elemen pertama rear digeser ke indeks berikutnya lalu elemen baru disimpan di array[rear] dan size akan bertambah. Setelah elemen berhasil ditambahkan maka akan muncul output item_3008 + “ enqueue to queue”.

5. Dequeue (menghapus elemen)

```
int dequeue_3008() {
    if (isEmpty_3008(this))
        return Integer.MIN_VALUE;
    int item_3008 = this.array_3008[this.front_3008];
    this.front_3008 = (this.front_3008 + 1) % this.capacity_3008;
    this.size_3008 = this.size_3008 - 1;
    return item_3008;
}
```

Gambar 2.2.1.5 dequeue

Saat ingin menampilkan lalu menghapus elemen yang ditampilkan pertama ambil elemen di front lalu geser front ke index berikutnya dan size akan berkurang.

6. Akses elemen depan dan belakang

```
int front_3008() {
    if (isEmpty_3008(this))
        return Integer.MIN_VALUE;

    return this.array_3008[this.front_3008];
}
int rear_3008() {
    if (isEmpty_3008(this))
        return Integer.MIN_VALUE;

    return this.array_3008[this.rear_3008];
}
```

Gambar 2.2.1.6 akses elemen

front_3008() → melihat elemen paling depan tanpa menghapus.

rear_3008() → melihat elemen paling belakang.

7. Display (menampilkan isi queue)

```
//mencetak elemen antrian
void display_3008() {
    int i_3008;
    if (front_3008 == rear_3008) {
        System.out.printf("\nAntrian Kosong");
        return;
    }
    //kunjungi dari belakang dan cetak
    for (i_3008 = front_3008; i_3008 < rear_3008; i_3008++) {
        System.out.printf("  %d <--", array_3008[i_3008]);
    }
    return;
}
```

Gambar 2.2.1.7 menampilkan isi queue

Menampilakan isi queue dari front ke rear.

2.2.2 Class QueueArrayDriver_2511533008

Kode Program :

```
package pekan4_2511533008;

public class QueueArrayDriver_2511533008 {

      public static void main(String[] args) {
            QueueArray_2511533008 queue_3008 = new
QueueArray_2511533008(1000);
            queue_3008.enqueue_3008(10);
            queue_3008.enqueue_3008(20);
            queue_3008.enqueue_3008(30);
            queue_3008.enqueue_3008(40);
            System.out.println("Item di depan: " +
queue_3008.front_3008());
            System.out.println("Item paling belakang: " +
queue_3008.rear_3008());
            System.out.println("tampilan queue");
            queue_3008.display_3008();
            System.out.println();
            System.out.println(queue_3008.dequeue_3008() + "
dihapus dari queue");
            System.out.println("Item di depan: " +
queue_3008.front_3008());
            System.out.println("Item di belakang: " +
queue_3008.rear_3008());
            System.out.println("tampilan queue setelah satu data
dihapus");
            queue_3008.display_3008();

      }

}
```

Penjelasan :

Pada class QueueArrayDriver_2511533008 kita menggunakan adt pada class QueueArray_2511533008 yang telah kita buat sebelumnya untuk membuat program berjalan.

1. Membuat objek queue_3008 dengan kapasitas 1000 elemen.

```
QueueArray_2511533008 queue_3008 = new QueueArray_2511533008(1000);
```

Gambar 2.2.2.1 membuat objek queue

2. Menambahkan elemen pada objek queue_3008.

```
queue_3008.enqueue_3008(10);
queue_3008.enqueue_3008(20);
queue_3008.enqueue_3008(30);
queue_3008.enqueue_3008(40);
```

Gambar 2.2.2.2 tambah elemen

3. Menampilkan elemen yang berada di depan dan paling belakang.

```
System.out.println("Item di depan: " + queue_3008.front_3008());
System.out.println("Item paling belakang: " + queue_3008.rear_3008());
```

Gambar 2.2.2.3 tampilkan elemen depan dan belakang

4. Menampilkan semua elemen pada queue.

```
System.out.println("tampilan queue");
queue_3008.display_3008();
```

Gambar 2.2.2.4 tampilkan elemen

5. Menghapus elemen paling depan

```
System.out.println(queue_3008.dequeue_3008() + " dihapus dari queue");
```

Gambar 2.2.2.5 hapus elemen

2.2.3 Class QueueLinkedList_2511533008

Kode Program:

```
package pekan4_2511533008;
import java.util.LinkedList;
import java.util.Queue;

public class QueueLinkedList_2511533008 {

	public static void main(String[] args) {
		Queue<Integer> q_3008 = new LinkedList<>();
		// tambah elemen {0, 1, 2, 3, 4, 5} ke antrian
		for (int i_3008 = 0; i_3008 < 6; i_3008++)
			q_3008.add(i_3008);
		//menampilkan isi antrian
		System.out.println("Elemen Antrian "+ q_3008);
		//untuk hapus kepala antrian
		int hapus_3008 = q_3008.remove();
		System.out.println("Hapus Elemen = " + hapus_3008);
		System.out.println(q_3008);
		//untuk melihat atrian terdepan
		int depan_3008 = q_3008.peek();
		System.out.println("Kepala Antrian = " + depan_3008);

		int banyak_3008 = q_3008.size();
		System.out.println("Size Antrian = " + banyak_3008);

	}
```

Penjelasan :

1. Struktur program

```
public static void main(String[] args) {
    Queue<Integer> q_3008 = new LinkedList<>();
```

Gamabar 2.2.3.1 struktur program

Queue<Integer> → memebuat antrian dengan tipe data integer.

LinkedList<>() → digunakan untuk mengimplementasikan queue.

2. Menambahkan elemen menggunakan method add()

```
// tambah elemen {0, 1, 2, 3, 4, 5} ke antrian
for (int i_3008 = 0; i_3008 < 6; i_3008++)
    q_3008.add(i_3008);
```

Gambar 2.2.3.2 tambah elemen

3. Menampilkan isi queue

```
//menampilkan isi antrian
System.out.println("Elemen Antrian "+ q_3008);
```

Gambar 2.2.3.3 tampilkan isi

4. Menghapus elemen depan dengan method remove()

```
int hapus_3008 = q_3008.remove();
System.out.println("Hapus Elemen = " + hapus_3008);
System.out.println(q_3008);
```

Gambar 2.2.3.4 hapus elemen

5. Melihat elemen depan dengan method peek()

```
//untuk melihat atrian terdepan
int depan_3008 = q_3008.peek();
System.out.println("Kepala Antrian = " + depan_3008);
```

Gambar 2.2.3.5 tampilkan elemen depan

6. Mengetahui jumlah elemen yang ada dalam queue dengan method size()

```
int banyak_3008 = q_3008.size();
System.out.println("Size Antrian = " + banyak_3008);
```

Gambar 2.2.3.6 size elemen

2.2.4 ReverseData_2511533008

Kode Program :

```
package pekan4_2511533008;
import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;
```

```java
public class ReverseData_2511533008 {

	public static void main(String[] args) {
		Queue<Integer> q_3008 = new LinkedList<Integer>();
		q_3008.add(1);
		q_3008.add(2);
		q_3008.add(3); // [1, 2, 3]
		System.out.println("sebelum reverse " + q_3008);
		Stack<Integer> s_3008 = new Stack<Integer>();
		while (!q_3008.isEmpty()) { // Q -> S
			s_3008.push(q_3008.remove());
		}
		while (!s_3008.isEmpty()) { // S -> Q
			q_3008.add(s_3008.pop());
		}
		System.out.println("sesudah reverse = " + q_3008); //
[3, 2, 1]


	}

}
```

Penjelasan :

1. Membalik queue dengan stack

```java
Stack<Integer> s_3008 = new Stack<Integer>();
while (!q_3008.isEmpty()) { // Q -> S
    s_3008.push(q_3008.remove());
}
```

Gambar 2.2.4.1 q → s

2. Memindahkan Kembali ke queue

```java
while (!s_3008.isEmpty()) { // S -> Q
    q_3008.add(s_3008.pop());
}
```

Gambar 2.2.4.2 s → q

2.2.5 Class IterasiQueue_2511533008

Kode Program ;

```java
package pekan4_2511533008;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.Queue;

public class IterasiQueue_2511533008 {
```

```java
        public static void main(String[] args) {
                Queue<String> q_3008 = new LinkedList<>();

                q_3008.add("Praktikum");
                q_3008.add("Struktur");
                q_3008.add("Data");
                q_3008.add("Dan");
                q_3008.add("Algoritma");
                Iterator<String> iterator_3008 = q_3008.iterator();
                while (iterator_3008.hasNext()) {
                        System.out.print(iterator_3008.next() + " ");
                }

        }

}
```

Penjelasan :

1. Menambahkan elemen pada queue

```java
q_3008.add("Praktikum");
q_3008.add("Struktur");
q_3008.add("Data");
q_3008.add("Dan");
q_3008.add("Algoritma");
```

Gambar 2.2.5.1 tambah elemen

2. Membuat iterator

```java
Iterator<String> iterator_3008 = q_3008.iterator();
```

Gambar 2.2.5.2 membuat iterator

3. Iterasi elemen queue

```java
while (iterator_3008.hasNext()) {
    System.out.print(iterator_3008.next() + " ");
}
```

Gambar 2.2.5.3 iterasi elemen queue

hasNext() → cek apakah ada elemen berikutnya

next() → mengambil elemen berikutnya dari queue

loop akan mencetak semua elemen secara berurutan.

2.3 Output Program

2.3.1 Output QueueArrayDriver_2511533008

```
Problems   @ Javadoc   Console ×   Declaration
<terminated> QueueArrayDriver_2511533008 [Java Applicatio
10 enqueue to queue
20 enqueue to queue
30 enqueue to queue
40 enqueue to queue
Item di depan: 10
Item paling belakang: 40
tampilan queue
  10 <--  20 <--  30 <--
10 dihapus dari queue
Item di depan: 20
Item di belakang: 40
tampilan queue setelah satu data dihapus
  20 <--  30 <--
```

Gambar 2.3.1 output 1

2.3.2 Output QueueLinkedList_2511533008

```
<terminated> QueueLinkedList_2511533008 [Java A
Elemen Antrian [0, 1, 2, 3, 4, 5]
Hapus Elemen = 0
[1, 2, 3, 4, 5]
Kepala Antrian = 1
Size Antrian = 5
```

Gambar 2.3.2 output 2

2.3.3 Output ReverseData_2511533008

```
<terminated> ReverseData_2511533008 [Jav
sebelum reverse [1, 2, 3]
sesudah reverse = [3, 2, 1]
```

Gambar 2.3.3 output 3

2.3.4 Output IterasiQueue_2511533008

```
<terminated> IterasiQueue_2511533008 [Java Application] C:
Praktikum Struktur Data Dan Algoritma
```

Gambar 2.3.4 output 4

BAB III
PENUTUP

3.1 Kesimpulan

Praktikum struktur data Queue memberikan pemahaman bahwa antrian adalah salah satu bentuk struktur data linear yang bekerja dengan prinsip FIFO (First In, First Out), di mana elemen yang pertama masuk akan menjadi elemen pertama yang keluar. Melalui implementasi menggunakan Array maupun LinkedList dari Java Collection Framework, kita dapat memahami perbedaan antara antrian statis dan dinamis serta kelebihan masing-masing. Praktikum juga memperlihatkan bagaimana operasi dasar seperti enqueue, dequeue, peek, dan display dijalankan, serta bagaimana kombinasi dengan struktur data lain seperti Stack dapat digunakan untuk membalik urutan data. Selain itu, penggunaan Iterator menunjukkan cara menelusuri isi antrian tanpa menghapus elemen.

DAFTAR PUSTAKA

[1] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6th ed. Wiley, 2014.

[2] I. Lim, *Diktat Struktur Data*. Bandung: Institut Teknologi Bandung, 2008.

[3] Universitas Andalas, "Modul Praktikum Struktur Data: Queue," Fakultas Teknologi Informasi, 2026.

TUGAS

1. Kode Program

1.1 Class AntrianLoket_2511533008

```
package pekan4_2511533008;

public class AntrianLoket_2511533008 {
    private int front_3008, rear_3008, max_3008;
    private String[] queue_3008;

    public AntrianLoket_2511533008(int kapasitas_3008) {
        max_3008 = kapasitas_3008;
        queue_3008 = new String[max_3008];
        front_3008 = -1;
        rear_3008 = -1;
    }

    public boolean isEmpty_3008() {
        return (front_3008 == -1 && rear_3008 == -1);
    }

    public boolean isFull_3008() {
        return (rear_3008 == max_3008 - 1);
    }

    public void enqueue_3008(String data_3008) {
        if (isFull_3008()) {
            System.out.println("Antrian penuh!");
        } else {
            if (isEmpty_3008()) {
                front_3008 = 0;
            }
            rear_3008++;
            queue_3008[rear_3008] = data_3008;
            System.out.println("Data berhasil ditambahkan ke antrian");
        }
    }

    public void dequeue_3008() {
        if (isEmpty_3008()) {
            System.out.println("Antrian kosong!");
        } else {
            System.out.println(queue_3008[front_3008] + " telah
dilayani");
            if (front_3008 == rear_3008) {
                front_3008 = rear_3008 = -1;
            } else {
                front_3008++;
            }
        }
```

```
    }

    public void display_3008() {
        if (isEmpty_3008()) {
            System.out.println("Antrian kosong!");
        } else {
            System.out.println("Isi antrian:");
            for (int i_3008 = front_3008; i_3008 <= rear_3008; i_3008++)
{
                System.out.println((i_3008 - front_3008 + 1) + ". " +
queue_3008[i_3008]);
            }
        }
    }

    public void reverse_3008() {
        if (isEmpty_3008()) {
            System.out.println("Antrian kosong!");
        } else {
            System.out.println("Isi antrian setelah dibalik:");
            for (int i_3008 = rear_3008; i_3008 >= front_3008; i_3008--)
{
                System.out.println((rear_3008 - i_3008 + 1) + ". " +
queue_3008[i_3008]);
            }
        }
    }

}
```

1.2 Class Queue_2511533008

```
package pekan4_2511533008;
import java.util.Scanner;

public class Queue_2511533008 {

    public static void main(String[] args) {
           Scanner sc_3008 = new Scanner(System.in);
        AntrianLoket_2511533008 antrian_3008 = new
AntrianLoket_2511533008(10);

        int pilihan_3008;
        do {
            System.out.println("=== PROGRAM ANTRIAN LOKET ===");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Hapus Antrian");
            System.out.println("3. Tampilkan Antrian");
            System.out.println("4. Reverse");
            System.out.println("5. Keluar");
            System.out.print("Pilih menu: ");
```

```
            pilihan_3008 = sc_3008.nextInt();
            sc_3008.nextLine();

            switch (pilihan_3008) {
                case 1:
                    System.out.print("Masukkan nama pelanggan: ");
                    String nama_3008 = sc_3008.nextLine();
                    antrian_3008.enqueue_3008(nama_3008);
                    break;
                case 2:
                    antrian_3008.dequeue_3008();
                    break;
                case 3:
                    antrian_3008.display_3008();
                    break;
                case 4:
                    antrian_3008.reverse_3008();
                    break;
                case 5:
                    System.out.println("Program selesai");
                    break;
                default:
                    System.out.println("Pilihan tidak valid!");
            }
        } while (pilihan_3008 != 5);

        sc_3008.close();
    }

}
```

Class AntrianLoket_2511533008 merupakan class adt yang dimana kita bisa memakainya pada class Queue_2511533008.

2. Output Program

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: Nada
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Nada

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: Naya
Data berhasil ditambahkan ke antrian
```

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 4
Isi antrian setelah dibalik:
1. Naya
2. Nada

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
Nada telah dilayani

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
Naya telah dilayani
```